

Troubleshooting

Every entry below is symptom -> check -> fix, grounded in the code that ships in this repo, not in how a hook is supposed to behave. Where a command is shown as `<crew>/hooks/scripts/...`, that is `${CLAUDE_PLUGIN_ROOT}/hooks/scripts/...` inside a command file, or the plugin's cache directory when you run it by hand — see the shorthand in [Memory and Obsidian](#).

Stale Install

Symptom: a command from this guide (`/crew:status`), a new hook, a config key) does not exist, or behaves like an older release.

- **Check the installed version against the repo.**

```
bash cat ~/.claude/plugins/installed_plugins.json | grep -A3 '"crew"'
```

Compare the commit or version it records against `plugin/crew/.claude-plugin/plugin.json`'s version in the marketplace clone. `claude plugin update` decides whether to re-copy a plugin by comparing the **declared version**, not its content — a content change that ships with no version bump means every already-installed copy reports "already at the latest version" forever, holding the old code, with nothing in the repo itself looking wrong (`CLAUDE.md`, "Stop and ask"). This class of bug is why `/crew:status`'s absence, or a hook that clearly predates a documented fix, is worth checking here before assuming the fix is wrong.

A related, sharper case: on a directory-source install (a dev checkout registered as a local plugin path rather than pulled from a marketplace), `installed_plugins.json` can name a version string with **no relationship to the code that actually executes** — the tree runs whatever is on disk regardless of what that file says (`TOD0.md`, "installed_plugins.json records 0.19.56 while 0.19.63 executes"). On that install shape, trust the code over the version string.

- **Fix:**

```
bash claude plugin update crew@useful-claude-add-ons
```

then **restart** claude — a running session does not pick up a re-copied plugin. If the update reports "already at the latest version" and you can see the fix in the marketplace repo's main, the version was not bumped when it should have been; that is a defect to file against the plugin, not something a client-side retry fixes.

- **Check it worked:**

```
/crew:status
```

Read-only, at most 40 lines, dispatches nothing. Its config line names which config file it read (`.crew/crew.json` for 1.0, `.crew/config.json` for 0.20); codemap reports behind: `<subsystem>` when a map's anchor and the path diff both say it is stale.

Hook Noise

Every hook crew registers, from `plugin/crew/hooks/hooks.json`, and the config key that turns its behaviour up or down. All events are registered once for bash and once for PowerShell (`shell: "powershell"`), so a Windows machine with both bash and pwsh on PATH fires both halves of every pair — that is what "even counts" in the file means, not a bug.

Event	Script	What it does	Switch	Default
SessionStart	handoff-read.sh	resets its once-per-session markers; reads back <code>.work/HANDOFF.md</code> only when <code>memory.inject</code> is false	— (always runs)	on
SessionStart	platform-sync.sh	detects/repairs platform. <code>{os,wsl,shell,windowsHostIp}</code>	— (always runs)	on
SessionStart / UserPromptSubmit / PostToolUse / SubagentStart	crew-context.sh	injects code-map slices and vault recall, tracks the token budget	<code>context.enabled</code> (whole hook), <code>memory.inject</code> (codemap, handoff and vault injection — on by default since 1.0.0)	<code>context.enabled</code> true, <code>memory.inject</code> true
UserPromptSubmit	approval-hook.sh	records or refuses a <code>/crew:approve</code>	always records; enforcement depends on	n/a

Event	Script	What it does	Switch	Default
		<id> receipt	scope.mode	
PreToolUse (Bash/PowerShell)	promote-gate.sh	the six command/prod uction guards plus mergeGate	guards.terraformApply, guards.forcePush, guards.adminMerge, guards.mergeGate, guards.cloudDestructive, guards.sqlDestructive, guards.prodDatabase, guards.prodServer	block / non strictest ti
PreToolUse (Write Edit)	role-write- guard.sh	refuses a write outside the dispatched role's declared scope	guards.roleWrites (block/report/off)	off
PreToolUse (Bash PowerShell)	cloud-guard.sh	destructive aws/az commands and wrong-identity commands	guards.cloudGuard (block/report/off)	off
PreToolUse (Write Edit MultiEdit NotebookEdit Bash PowerShell)	scope-guard.sh	plan-approval + ticket scope guard	scope.mode (off/report/block/auto), scope.allowCliApproval	off, false
PreCompact	handoff-write.sh	writes the handoff note before compaction	context.autoWrapUp, context.handoffPath	on
Notification	notify.sh	pings an external channel	notify.provider (none disables it),	none

Event	Script	What it does	Switch	Default
			notify.events	
Stop	verify-gate.sh	runs .crew/verify.js on's checks	verifyGate (boolean)	true
Stop	context-watch.sh	nags for a handoff near the context budget, drives auto-clear	context.enabled, context.warnAt, context.budgetTokens, context.reserveTokens, context.autoClear.*	on, warnAt
Stop	completion-audit.sh	diffs the whole tree against the ticket's scope base	scope.mode	off

- **Silence one hook without touching the rest:** set its own key. `guards.roleWrites: off`, `guards.cloudGuard: off` and `scope.mode: off` are already the shipped defaults — a noisy session usually means one of these was turned on somewhere (repo or machine-global) and forgotten, not that the default changed.
- **Silence the SessionStart context specifically:** `{"memory": {"inject": false}}` stops the context hook; `handoff-read.sh` then prints the handoff on resume instead.
- **See what the context hook is actually doing:**

```
bash python3 "<crew plugin root>/hooks/scripts/crew_context.py" --stats
```

Reports emissions, injected characters, dedup hits, and vault-recall hit/miss/skip counts with a reason breakdown (see "Obsidian" below). The log itself is `<git-common-dir>/crew/context-log.jsonl` (or `.work/crew/context-log.jsonl` outside git), shared across every worktree of the repo. - **A hook fails and you cannot tell why:** every wrapper here resolves its own Python (python3/python/py) rather than assuming one is present, and every one that can block fails **closed** — never silently — when no interpreter resolves. See "no-python fail-closed" under Scope, below; the same shape applies to `scope-guard.sh`, `role-write-guard.sh`, `cloud-guard.sh` and `verify-gate.sh`.

Review Loops

Each ticket gets **two** review rounds, reserved before the reviewer runs, tracked in `<git-common-dir>/crew/review/<ticket>.json` — shared by every worktree of the repo, so a second worktree of the same repo spends the same budget (`review_ledger.py`).

- **Symptom: review keeps re-running / "budget exhausted".** **Check:** `bash python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --status` A round's outcome is CLEAN, FINDINGS or INCOMPLETE. A completed round 2 — either FINDINGS or INCOMPLETE — leaves the ticket state REVIEWED, so its FINDINGS can still be accepted. A **third** reservation attempt is refused outright and the state becomes NEEDS_REPLAN; that refusal, and an explicit `--reject`, are the only two ways into NEEDS_REPLAN. **Fix:** own the FINDINGS with `--accept --by <who>` (only the most recent completed round, only once, never once NEEDS_REPLAN), or write a new plan and get it approved — `crew_ticket.py` approve on a NEEDS_REPLAN ticket opens a fresh budget of two rounds counted from the successor plan; the rounds already spent stay in the ledger and are not erased.
- **Symptom: /crew:done refuses with a receipt error.** **Check:** `bash python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --check-receipt` A CLEAN verdict writes a receipt automatically; a FINDINGS verdict only becomes one through `--accept`. `--check-receipt` rebuilds the review bundle from the receipt's recorded base and fails unless the hash still matches, the receipt is for the **latest** recorded round, and the state is not NEEDS_REPLAN — so editing a file after the reviewer read it, or after the receipt was written, invalidates the receipt even though nothing about the ledger itself looks wrong. **Fix:** if the edit was deliberate, get the ticket reviewed again (spends the next round); if it was accidental, revert the edit and re-check.
- **Symptom: you want to send a ticket back without spending the third round.** `bash python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --reject --by <who>` Refuses on a ticket already ACCEPTED or already NEEDS_REPLAN.

Scope: Approval and the Completion Audit

A ticket's contract lives in `.work/tickets/<id>/: spec.md`'s `## Touch` section names every path the ticket may change; `plan.md`'s `Files:` lines must each fall inside `Touch`. Two hooks hold a session to that contract — see [Daily workflow: scope and approval](#) for the contract itself. This section is what goes wrong with the approval and the audit.

- **Symptom: an edit inside Touch is still refused. Check:** approval status. `bash python3 "<crew>/hooks/scripts/crew_ticket.py" status --ticket <id>` Returns approved (both `spec.md` and `plan.md` hash to what was approved), stale (either file changed since), or none. **Editing `spec.md` or `plan.md` after approval makes the approval stale immediately** — that is deliberate: an edited spec cannot silently widen what the guard accepts. Only a receipt from the user's own prompt counts by default; a `cli`-sourced receipt (written by `crew_ticket.py approve` directly, for tests/CI) is accepted only when `.crew/config.json` sets `scope.allowCliApproval: true`. **Fix:** re-approve. The user types `/crew:approve <id>` again — nothing else can write the receipt; `scope_guard.py` refuses a Write/Edit under `<git-common-dir>/crew/` in every mode but `off`, so a session cannot forge or refresh its own approval.
- **Symptom: you need to touch one more path mid-ticket. Fix:** amend `spec.md`'s `##` Touch (widen it), then approve again. There is no partial-approve; amending scope is edit-then-approve, same as any other spec change.
- **Symptom: `/crew:done` refuses on "out of scope" for a file the edit guard never saw.** This is the Stop-time completion audit (`completion_audit.py`), not the edit guard. The edit guard only sees Write/Edit/MultiEdit/NotebookEdit tool calls; a `sed -i`, a redirect, a formatter or a `git mv` never reaches it. The audit instead diffs the **whole working tree** against the ticket's scope base (`scope_base.resolve` — the commit the ticket started from) across committed, staged, unstaged and untracked changes, so a shell-made write is caught here even though nothing blocked it at the time. **Check it directly, without waiting for a Stop:** `bash python3 "<crew>/hooks/scripts/completion_audit.py" --check --ticket <id>` **Fix:** either the path genuinely needs to be in scope (widen Touch and re-approve), or revert the out-of-scope change. Files `git` ignores (including everything under `.crew/`) and `.work/` itself are outside what the audit can see at all — that is by design, not a gap to work around.
- **`scope.mode` values, and what "auto" means:** `off` (hooks do nothing, the default), `report` (allows everything, logs the row to `.crew/guard.log`), `block` (refuses out-of-scope writes and fails the completion audit), `auto` (report for the first ten tickets approved in this repo, then `block`). A config file that exists but fails to parse, or names an unknown mode, resolves to `block` and says so — the unknown never collapses into the permissive value.
- **No-python fail-closed, and why it is not a bug:** if none of `python3/python/py` resolves to a *proved* working interpreter, `scope-guard.sh` (and `completion-audit.sh`, byte-for-byte the same logic) does **not** silently allow the write. It checks whether `.crew/config.json`

provably sets `scope.mode` to `off` — a crude, non-JSON textual check, because it runs in exactly the situation where python cannot parse the file for it — and only then exits 0. Anything it cannot prove off (a corrupt file, report, auto, block, or no scope key at all) fails **closed** with exit 2: "no usable python ... failing closed". Fix by installing a real Python 3, not by reading the closed refusal as a false positive.

Cloud Guard False Positives

`cloud-guard.sh / cloud_guard.py`, a PreToolUse hook on Bash and PowerShell. **Ships off** (`guards.cloudGuard: off`); turn it on with `{"guards": {"cloudGuard": "report"}}` first to see what it *would* refuse before enforcing with "block".

- **SQL text that merely looks destructive is not blocked.** DROP/TRUNCATE inside a string literal or a comment does not count: the guard strips literals and comments before matching, read both as standard SQL and as MySQL, because the two dialects disagree about what `\'` and `--` mean — a payload is flagged only if **either** reading still shows the keyword outside a literal. So `psql -c "SELECT 'DROP TABLE x' AS s"` and `psql -c 'SELECT 1 -- DROP TABLE x'` are both allowed (`plugin/crew/tests/test_cloud_guard.py`, cases `sql-literal/sql-comment/sql-block-comment`). If a query like this is still blocked, the reason (`[sqlDestructive]` in the deny message) is worth reading closely — the keyword may genuinely sit outside every literal reading.
- **A dry-run, help, or -WhatIf invocation is still classified as destructive.** The guard matches on the *subcommand*, not on flags that make it a no-op: `terraform destroy --help`, `aws ec2 terminate-instances --dry-run` and `Remove-AzResourceGroup -WhatIf` are all judged exactly like the command they simulate, because nothing in `_terraform_destructive/_aws_destructive/the Remove-Az*` matcher inspects those flags. This is a known gap, not a config bug — there is no key that special-cases them. If you need to run one of these unattended, use report mode while testing, or approve the one-shot marker the deny message names.
- **An "unpinned" repo makes an ordinary destructive command ask or deny, even when it is obviously fine.** With no `cloud.*` pins, a read-only command (`aws s3 ls`, `az group list`) runs unchecked — but *any* destructive command has an identity the guard cannot vouch for, which is **unknown**, and unknown is never allowed unattended: it **asks** when a person is there to answer and is **denied** when not (`CREW_UNATTENDED=1`, CI, or a non-interactive session). This is the rule most likely to surprise a first-time user, because

it fires even in a repo that has never configured anything cloud-related. **Fix — pin the identity this repo may act as**, in `.crew/config.json` (repo-only; a machine-global cloud block is ignored): `json { "cloud": { "awsProfiles": ["acme-dev", "acme-sandbox-*"], "awsRegions": ["eu-west-1"], "azureSubscriptions": ["00000000-0000-0000-0000-000000000000", "Acme Dev"] } }` A profile/region/subscription outside the pins is denied outright (`[cloudIdentity]`). Static `AWS_ACCESS_KEY_ID` credentials and `Remove-Az*`'s saved PowerShell context are **always** unknown — the guard has no way to read either — so pinning does not silence those; only removing the static keys or naming the account through `--profile/--subscription` does.

Obsidian

See [Memory and Obsidian](#) for setup. What goes wrong day to day:

- **Symptom: nothing is being captured / gardened. Check:** is the *primary* vault reachable? Capture, import and the gardener write to the primary vault only, and **never** fall back to a recall vault. If the primary is unavailable — an unmounted drive, a renamed folder — they write nothing and say why on stderr; the session itself is never blocked. `$V0 adopt` shows which vault is currently primary.
- **Symptom: recall never surfaces anything you know is in the vault. Check:** `bash python3 "<crew plugin root>/hooks/scripts/crew_context.py" --stats` Read the miss reasons line. `cli-missing` means `obsidian-vault` is not installed where `crew` can find it; `cli-exit-N`, `cli-timeout`, `cli-bad-json` mean the CLI ran and failed; `no-vaults` means no vault is currently primary or recall; `no-hits` means the search itself found nothing. Only `no-hits` means "the vault genuinely has nothing relevant" — every other reason is a setup problem. Also confirm `memory.inject` is not false in `.crew/config.json` — recall and code-map injection are on by default since 1.0.0, so "no recall ever appears" with the key set to false is expected. **Fix:** `$V0 recall --query "<words you expect>"` directly, to separate "the CLI can't find it" from "the hook isn't calling the CLI".
- **Symptom: the gardener seems to be falling behind.** It runs in bounded passes on purpose, not continuously: **at most 5 items or 10 minutes per run**, whichever comes first, on **one designated host**, one run at a time. A session whose transcript lives on another machine is left queued for that machine and does not spend one of the five slots. **Check:** `$V0 queue` (what is waiting) and `~/.claude/obsidian/gardener.log` for lines starting `left queued`. **Fix:** run a bounded pass by hand (`/obsidian-vault:garden` or `$V0`

garden-run), or drain a large backlog in batches: `$V0 drain --apply --batches 4`.

- **Symptom: two machines sharing one vault seem to be racing each other.** They are not writing the same file: each host appends to its own `inbox/pending-reflect.<host>.md` and acknowledges to its own `inbox/reflected.<host>.md`, so two machines syncing one vault never contend for the same queue file. An older `inbox/pending-reflect.md` (no host suffix) is still read if present, so an old backlog drains rather than being stranded — it is just never written again.

Auto Wrap-Up, Clear and Resume

See [Auto wrap-up, clear and resume](#) for the full cycle that lets a long session finish itself cleanly, clear, and pick up where it stopped — what wrap-up, auto-clear and resume each do, the machine-global keys that turn auto-clear on, and every reason a clear can refuse (logged to `.crew/.autoclear.log`).

Turning Things Off

Every switch named above, in one place. "Off" for a guard means the `PreToolUse` hook still fires but returns immediately without judging anything; "off" for `verifyGate` means the `Stop` hook does nothing at all that turn.

Key	Lives in	Values	What "off"/floor means
<code>memory.inject</code>	repo only	bool	false: no code-map, handoff or vault text is injected (default true since 1.0.0)
<code>guards.cloudGuard</code>	both layers, ratchets	block/ report/off	off: the hook reads this key and exits
<code>guards.roleWrites</code>	both layers, ratchets	block/ report/off	off: every Write/Edit is allowed unconditionally

Key	Lives in	Values	What "off"/floor means
<code>guards.terraformApply</code> , <code>forcePush</code> , <code>adminMerge</code> , <code>mergeGate</code> , <code>cloudDestructive</code> , <code>sqlDestructive</code>	both layers, ratchets	block/ask/ allow	there is no "off" — allow is the most permissive tier, still logged
<code>guards.prodDatabase</code> , <code>guards.prodServer</code>	both layers, ratchets	none/read/ full	none is both the default and the floor
<code>scope.mode</code>	repo only	off/report/ block/auto	off: neither the edit guard nor the completion audit runs
<code>scope.allowCliApproval</code>	repo only	bool	false: only a /crew: approve typed by the user counts
<code>verifyGate</code>	repo only	bool	false: the Stop verify gate does not run at all
<code>context.enabled</code>	both layers	bool	false: <code>context-watch.sh</code> (the Stop nag) does nothing
<code>context.autoClear.enabled</code> , <code>autoWrapUp</code>	both layers	bool	each independently disables one leg of the wrap-up/clear loop
<code>notify.provider</code>	both layers	provider name / none	none: the Notification hook sends nothing
<code>change.requireForProduction</code>	both layers, ratchets (may only turn ON)	bool	false: promoting needs no approved change request
<code>install.policy</code>	both	see	narrowest tier refuses

Key	Lives in	Values	What "off"/floor means
	layers, ratchets	plugin/crew /CONFIG.md \$"install"	more install actions

~/.claude/crew/config.json is the machine-global layer; .crew/config.json is per-repo and wins where both speak. A **ratcheted** key (marked above) can only be *narrowed* by the repo relative to the machine-global value, never widened — a repo cloned from someone else cannot silently loosen a guard the machine owner set to block. /crew:config shows where each setting actually came from.

Windows Notes

- **pwsh is not on Git Bash's PATH.** Every script that shells out to PowerShell names it by an absolute path or resolves it explicitly; a bare pwsh in a hand-typed command fails as "command not found", which is easy to misread as PowerShell itself being broken.
- **Git Bash ships without python3.** Every hook wrapper here resolves python3, then python, then py, and fails loudly on stderr rather than silently standing down when none resolves (see "no-python fail-closed", above). If a hook you expect to run says nothing at all, check for a usable Python on PATH first.
- **Both flavours are always registered.** hooks.json pairs every bash command with a shell: "powershell" sibling on the same event, so a machine with both bash and pwsh present runs both. This is not double-firing to fix — it is how the same hook reaches both shells; each PowerShell twin stands itself down on the wrong platform (\$env:OS -ne 'Windows_NT') rather than the bash twin doing that job.

Auto Wrap-Up, Auto-Clear and Auto-Resume

This section is written for the troubleshooting guide (troubleshooting.md, arriving in T10), which should link to it. It covers the cycle that lets a long session finish itself cleanly, clear, and pick up where it stopped.

What the Cycle Does

1. **Wrap-up.** When the context window crosses its threshold, the Stop hook blocks once. It asks the session to finish or park the change in flight, write `.work/HANDOFF.md`, and update the ticket. This happens once for each threshold crossing in each session. It never happens on the forced continuation that follows a block.
2. **Clear.** After the handoff is written, auto-clear types `/clear` into the terminal that runs this session. It does this only when all four of these hold: - this machine opted in; - the handoff was written by this session after the wrap-up request, is longer than a stub, and is not the automatic PreCompact skeleton; - the context reading that caused the wrap-up was a real measurement (see "When it will not clear"); - the target window can be identified exactly.
3. **Resume.** The new session starts with the handoff and its next action already in context, within the 3,000-character resume budget. Nothing starts working unprompted: crew does not use `initialUserMessage`, so you press Enter to continue.

Turning It on and Off

Wrap-up and resume are on by default. Auto-clear is **off** by default. You turn it on per machine, in the machine-global config, because it drives this machine's keyboard:

Where	Key	Value	Effect
~/ .claude/ crew/ config.json (Windows: %USERPROFILE% \ .claude\crew\ config.json)	context.autoClear.enabled	true	turns auto-clear on for every crew repo on this machine
same	context.autoClear.method	"auto" (default), "tmux", "xdotool", "windows", "none"	how the keystroke is delivered
same	context.autoClear.windowTitle	a	fallback when the

Where	Key	Value	Effect
		substring of the terminal's title	window cannot be found by process id
same	<code>context.autoClear.delaySeconds</code>	3	wait before typing, so the prompt exists
same	<code>context.autoClear.minHandoffLines</code>	5	a shorter handoff counts as a stub
<code>.crew/ config.json</code> (the repo)	<code>context.autoClear.enabled</code>	false	switches auto-clear off in this repo. A repo can switch it off, but true here does not switch it on.
<code>.crew/ config.json</code>	<code>context.autoWrapUp</code>	false	replaces the wrap-up instruction with a plain handoff request. It still blocks once.
<code>.crew/ config.json</code>	<code>context.enabled</code>	false	turns off the whole context watcher, wrap- up included
<code>.crew/ config.json</code>	<code>memory.inject</code>	false	handoff - read prints the handoff for you to read, without extracting the next action. <code>context.autoResume</code> is no longer read.

`/crew:config` shows where each value comes from and walks you through the global file. To turn the cycle off on a machine, set `context.autoClear.enabled` to false or delete the key.

When It Will Not Clear

Each refusal is logged with its reason to `.crew/.autoclear.log`. A Stop hook's stderr is not shown to you, so check that log first.

Log says	Meaning
<code>not trustworthy (estimated-from-transcript-size)</code>	no usage record was found, so the reading was an estimate
<code>not trustworthy (unknown-window)</code>	the model's context window is unknown and <code>context.budgetTokens</code> is not set
<code>not trustworthy (stale-reading-before-compaction)</code>	the last reading came from before a compaction
<code>the wrap-up marker is empty or not JSON</code>	the marker was not written by this version of the hook. Nothing clears on it.
<code>predates this session's wrap-up request</code>	the handoff on disk is older than the request, probably left by an earlier session
<code>automatic PreCompact skeleton</code>	only the compaction skeleton exists, not a real handoff
<code>N windows have a title containing .../ has N windows</code>	the window is ambiguous, so nothing is typed
<code>no window belongs to any ancestor of this hook</code>	the terminal's window could not be found by process id, and no <code>windowTitle</code> is set
<code>tmux pane %N could not be confirmed</code>	<code>\$TMUX_PANE</code> is not the pane that runs this session
<code>method wtype ... is refused</code>	Wayland's <code>wtype</code> cannot target a window. Use <code>tmux</code> instead.

Two sessions in one repository no longer interfere with each other. Wrap-up markers are `.crew/.handoff-requested-<session_id>`, and a `SessionStart` clears only its own session's marker.

Which Window Gets the Keystroke

- **tmux (Linux, macOS, WSL):** the pane in `$TMUX_PANE`, and only when that pane's process is an ancestor of the hook. No title is needed. This is the most reliable method.
- **X11 (xdotool):** the one visible window owned by the nearest ancestor process of the hook. If that process owns more than one window, `windowTitle` has to narrow the choice to one. If no ancestor owns a window, exactly one window must match `windowTitle`. The keystroke goes to that window id after it is activated and confirmed active.
- **Windows (SendKeys):** chosen the same way, by owner process first and then by title. After the delay, `/clear` is sent only when that exact window handle has focus.

Windows Terminal runs every tab in one window. A process-id match cannot tell which tab is active, so on Windows Terminal set `windowTitle` to something only this session's tab shows.